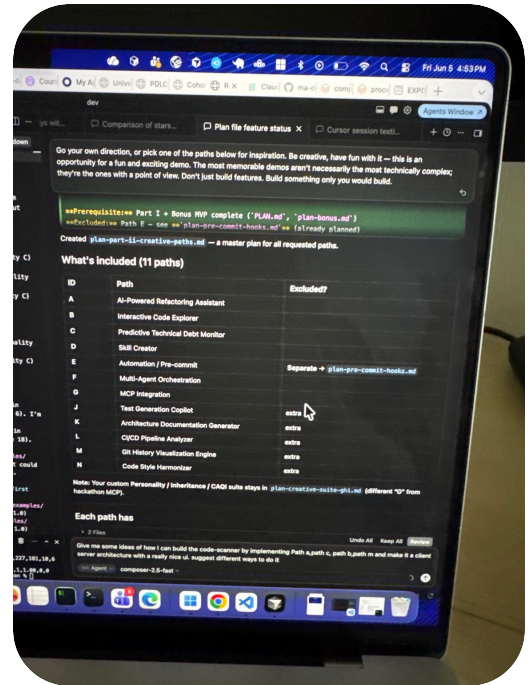
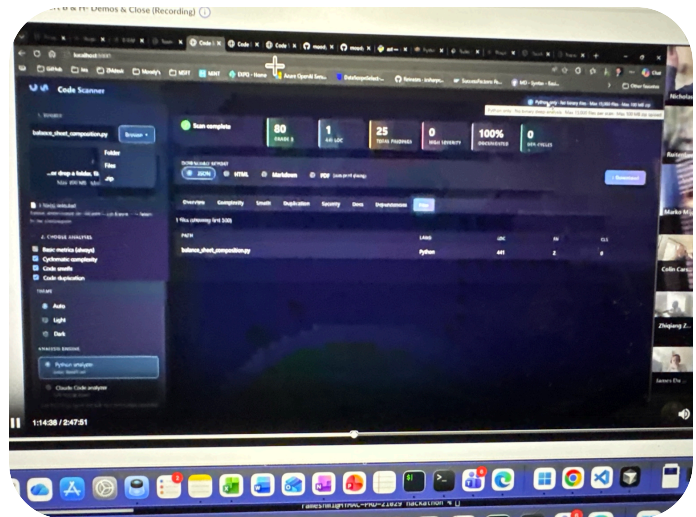
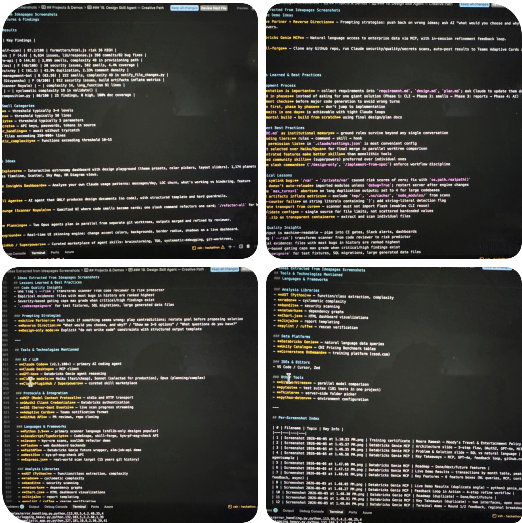










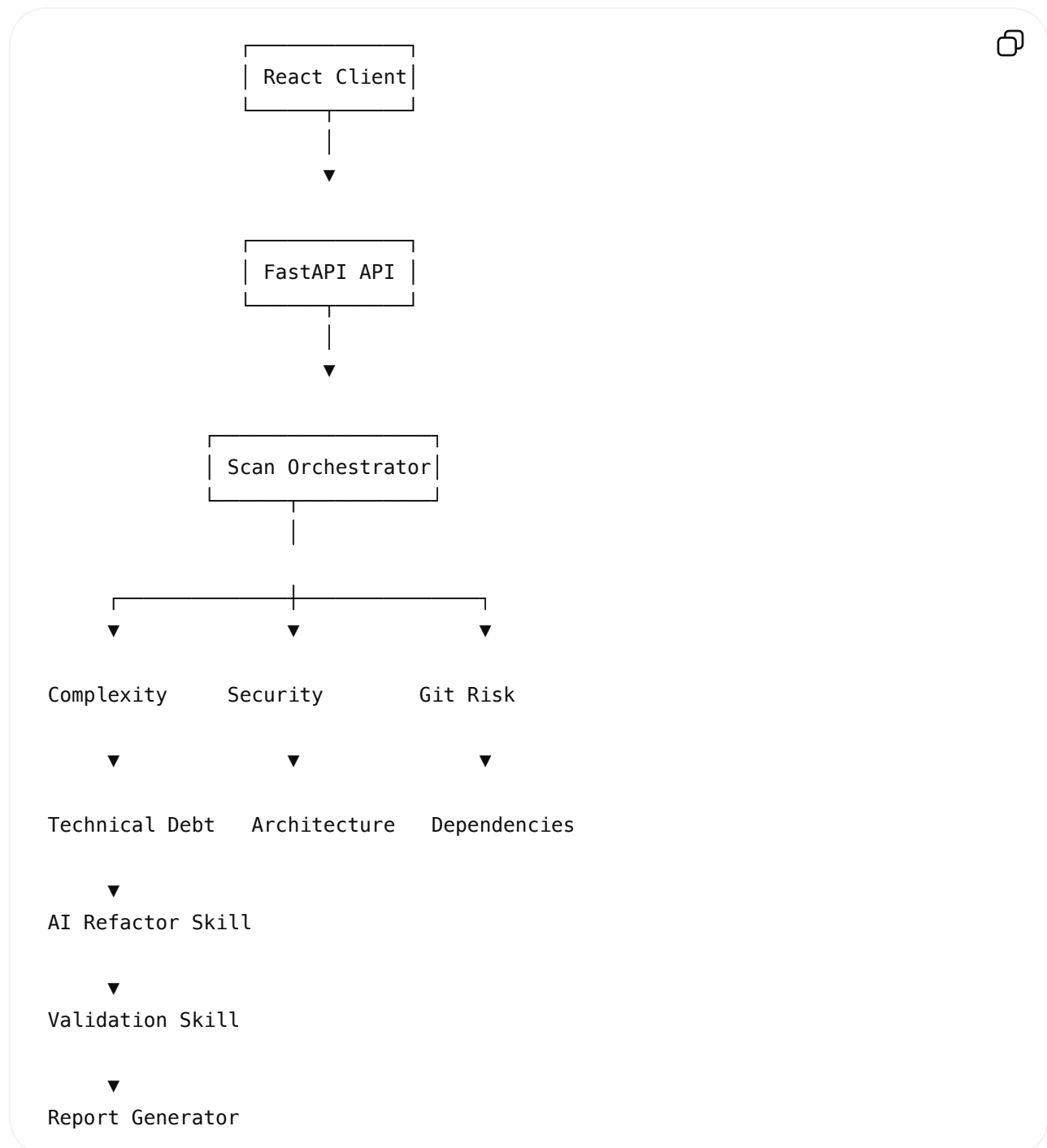
After reading through all the screenshots, there are really **three categories** of ideas:

- 1. Code Scanner Features**
- 2. AI Agent / MCP Features**
- 3. Architecture Patterns & UX Ideas**

If I were building a single project that combines everything, I would use a **Client → Server → Agent → Analyzer → Refactor → Validate** architecture.

Final Product Vision

CodePulse AI Platform



ALL IDEAS FOUND IN THE SCREENSHOTS

1. Repository Upload

Upload File
Upload Folder
Upload Zip
GitHub Repository URL



2. Drag and Drop Upload

Drop Repository Here



Supported:

Java
Python
JavaScript
TypeScript
SQL
Zip



3. Language Detection

Found repeatedly.

Detect:

Python
Java
JavaScript
TypeScript
Go
Rust
PHP
SQL
HTML
CSS
JSON
YAML



4. File Traversal Engine

Walk entire repository



Skip:

```
.git  
node_modules  
venv  
__pycache__  
dist  
build
```

5. Complexity Analyzer

Measure:

Cyclomatic Complexity
Cognitive Complexity
Function Length
Class Length



Common thresholds:

Complexity > 10

Function > 50 lines



6. Code Smell Analyzer

Found in almost every project.

Detect:

Long Function

Deep Nesting

Long Parameter List

God Class



God File

Dead Code

Unused Variables

7. Security Analyzer

Detect:

Hardcoded Secrets



API Keys

Passwords

Tokens

eval()

pickle.loads()

shell=True

SQL Injection

Unsafe Subprocess

8. Documentation Analyzer

Check:

Missing Docstrings



Missing Comments

Undocumented Public APIs

9. Dependency Analyzer

Find:

Circular Dependencies

Duplicate Libraries

Outdated Packages

Unused Packages

10. Architecture Explorer

This was one of the coolest ideas.

Generate:

Frontend

API

Services

Repositories

Database

Visual graph.

11. Interactive Code Explorer

Click:

Class

Method

File

View:

Dependencies

Call Graph

Imports

Related Files

12. Git History Intelligence

Nicolas' idea.

Use:

Commit Frequency



Bug Fix Count

Churn

Recency

Bus Factor

Formula:

Risk Score



=
Quality
+
Churn
+
Bug Fixes
+
Recency
+
Bus Factor

13. Technical Debt Predictor

Predict:

Future Problem Files



Future Refactoring Needs

High-Risk Modules

14. Risk Heatmap

Low Risk



Medium Risk

High Risk

Visualized by file.

15. Quality Scoring Engine

Generate:

Score 0-100



Letter Grade

A

B

C

D

F

16. AI Refactor Skill

Found in multiple demos.

Detect:

Long Method



Suggest:

Extract Method



Detect:

Nested If



Suggest:

Guard Clauses



Detect:

Hardcoded Secret



Suggest:

Environment Variable



17. Refactor Lounge

Scanner Royale idea.

Every issue becomes:

Card



Issue

Risk

Fix Button

18. One Click Refactor

Refactor File



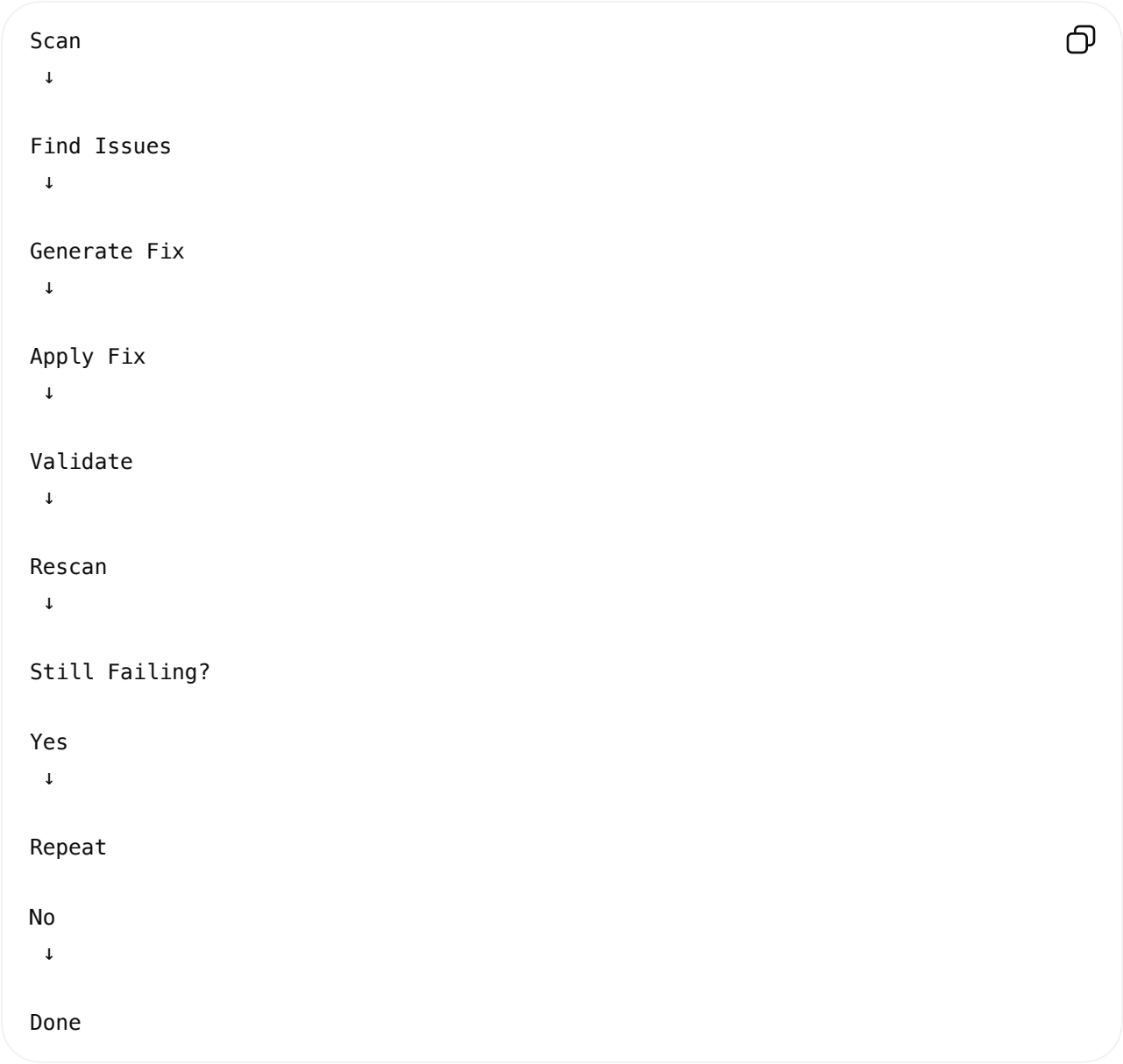
Refactor Function

Refactor All

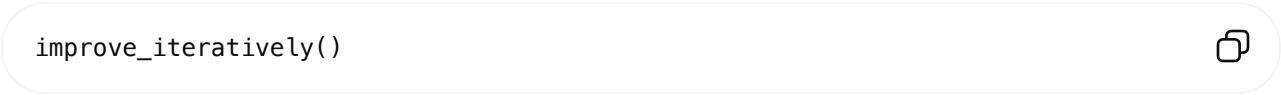
19. Iteration Until Clean Skill

This appeared several times.

Agent Loop



MCP Tool



or

```
fix_until_clean()
```



20. Validation Skill

After AI changes code:

Run:

Linters



Unit Tests

Static Analysis

Security Analysis

Verify:

No Regression



21. Test Generator

Generate:

Unit Tests



Integration Tests

Edge Cases

22. Documentation Generator

Generate:

README



Architecture Docs

API Docs

Refactor Plan

23. Executive Summary Generator

For managers.

Output:

Repository Grade



Top Risks

Recommendations

Estimated Improvement

24. MCP Server

Expose tools:

scan_project



scan_file

get_health_score

explain_finding

generate_refactor

fix_until_clean

download_report

25. Multi-Agent System

From Michael's project.

Planner Agent



Scanner Agent

Risk Agent

Refactor Agent

Validation Agent

Documentation Agent

Report Agent

Vishal / Dashboard Style UI

Based on the scanner dashboard screenshot:

CodePulse AI



Upload Repo

[Browse]

OR

[Drag & Drop]

GitHub URL

Features

- ☒ Complexity
- ☒ Security
- ☒ Architecture
- ☒ Git Risk
- ☒ Technical Debt
- ☒ Refactoring
- ☒ Tests
- ☒ Documentation

[Run Scan]

Grade A-

Risk 83

Security 2 High

Debt 14

Coverage 89%

Tabs

Overview

Architecture

Findings

Git Risk

Technical Debt

Refactor Lounge

Documentation

Reports

Download

PDF

HTML

JSON

Markdown

If I were building this for the hackathon

Must Have

- Client-server architecture
- Upload repo/folder/file
- Dashboard
- Download reports
- Complexity

- Security
- Technical debt

Differentiators

- Git-history risk prediction
- Architecture explorer
- AI refactor skill

Standout Feature

Refactor Until Clean

Scan

↓

Refactor

↓

Validate

↓

Rescan

↓

Repeat Until Grade A

That single workflow combines the strongest ideas from Nicolas, Advaitha, Tim, Scanner Royale, Michael's orchestration framework, MCP integration, and the dashboard UI into one coherent product.



also include the usecases and the architecture decisions that are already in place with a precommit workflow

Yes — add these sections to your design doc.

Use cases to include

1. Developer uploads a repo/file and runs selected scanners.

2. Developer drags and drops a file or ZIP for quick analysis.

3. Developer selects only the features they need:
complexity, security, dependency, git risk, refactor, docs.
4. Developer downloads scan output as PDF, HTML, JSON, or Markdown.
5. Developer uses Refactor Skill to fix high-risk findings.
6. Developer runs Iteration Until No Errors Skill:
scan → fix → validate → rescan → repeat.
7. Team uses pre-commit workflow to block risky code before commit.
8. Manager downloads Executive Summary for repo health and risk.
9. CI/CD uses JSON output as a quality gate.
10. Claude/Cursor uses MCP tools to call scanner features directly.

Architecture decisions already in place

1. Client-server architecture

Decision:

React client + FastAPI/Flask backend.

Reason:

Keeps UI separate from scanner logic and allows CLI/MCP reuse.

2. Pure scanner core

Decision:

Scanner core must not depend on Flask/FastAPI.

Reason:

Same scanner can be used by web app, CLI, MCP server, and pre-commit hook.

3. Plugin/analyzer architecture

Decision:

Each analyzer is registered separately.

Examples:

ComplexityAnalyzer

SecurityAnalyzer

DependencyAnalyzer

GitRiskAnalyzer

DocumentationAnalyzer

RefactorAnalyzer

Reason:

Easy to add new scan features without rewriting the scanner.

4. Feature selection from UI

Decision:

User selects which analyzers to run.

Reason:

Fast scans for small checks, deep scans for full review.

5. Refactor skill

Decision:

AI-generated fixes are handled as a separate skill.

Reason:

Scanning and fixing are different responsibilities.

6. Iteration until no errors skill

Decision:

Run loop:

scan → refactor → validate → rescan → repeat.

Reason:

Shows measurable improvement and prevents unsafe fixes.

7. Report generator

Decision:

Generate JSON first, then render PDF/HTML/Markdown.

Reason:

JSON is machine-readable for CI/CD; HTML/PDF/Markdown are human-readable.

8. Git-history risk scoring

Decision:

Risk combines static quality + churn + bug fixes + recency + bus factor.

Reason:

Files that change often and break often are usually higher risk.

9. MCP-ready tool layer

Decision:

Expose scanner actions as callable tools.

Examples:

```
scan_project
scan_file
get_health_score
explain_finding
generate_refactor
fix_until_clean
download_report
```

Reason:

Claude Code, Claude Desktop, Cursor, or other agents can reuse scanner abilities

10. Pre-commit workflow

Decision:

Run lightweight checks before commit.

Reason:

Stop obvious issues early before they enter Git history.

Pre-commit workflow

Developer edits code

↓

git commit

↓

pre-commit hook runs scanner

↓

Quick checks run:

- secrets
- syntax
- complexity threshold
- long functions
- dangerous calls

↓

If pass:

commit allowed

↓

If fail:

commit blocked with report



Pre-commit command idea

```
codepulse scan --staged --mode quick --fail-on high
```

Pre-commit config idea

```
repos:
  - repo: local
    hooks:
      - id: codepulse-scan
        name: CodePulse AI Pre-Commit Scan
        entry: codepulse scan --staged --mode quick --fail-on high
        language: system
        pass_filenames: false
```

Final architecture flow

```
React UI
  ↓
Backend API
  ↓
Scan Orchestrator
  ↓
Analyzer Plugins
  ↓
Risk Engine
  ↓
Refactor Skill
  ↓
Validation Skill
  ↓
Report Generator
  ↓
Download Center
```



Use this as the final story:

CodePulse AI lets developers upload code, run selected quality checks, generate AI refactors, validate fixes, repeat until clean, enforce checks before commit, and download reports for developers, managers, and CI/CD.



Using the following features given below combine and give me all the features in one go
there are really three categories of ideas:

1. Code Scanner Features
2. AI Agent / MCP Features
3. Architecture Patterns & UX Ideas

If I were building a single project that combines everything, I would use a Client → Server → Agent → Analyzer → Refactor → Validate architecture.

Show more ▾

Thought for a couple of seconds >

Below is the combined **all-in-one feature set** for your project.

CodePulse AI — Complete Feature List

1. Core Product

CodePulse AI is a client-server code intelligence platform that lets developers upload a repo, folder, ZIP, or file, choose analysis features, run scans, generate AI refactors, validate fixes, repeat until clean, enforce pre-commit checks, and download reports.

```
React UI
  ↓
FastAPI / Flask Backend
  ↓
Scan Orchestrator
  ↓
Analyzer Plugins
  ↓
Risk Engine
  ↓
AI Refactor Skill
  ↓
Validation Skill
  ↓
Report Generator
```



↓
Download Center

2. UI Features

Source Input

- Upload File
- Upload Folder
- Upload ZIP
- GitHub Repository URL
- Drag and Drop Upload
- Browse Repo
- Browse File



Supported Inputs

- Java
- Python
- JavaScript
- TypeScript
- SQL
- Go
- Rust
- PHP
- HTML
- CSS
- JSON
- YAML
- ZIP



Feature Selection

- ☒ Complexity Analysis
- ☒ Code Smell Analysis
- ☒ Security Scan
- ☒ Documentation Scan
- ☒ Dependency Analysis
- ☒ Architecture Explorer
- ☒ Interactive Code Explorer
- ☒ Git History Risk
- ☒ Technical Debt Prediction



- ☒ AI Refactoring
- ☒ Test Generation
- ☒ Validation
- ☒ Executive Summary
- ☒ MCP Export

Dashboard Tabs

- Overview
- Architecture
- Findings
- Git Risk
- Technical Debt
- Refactor Lounge
- Tests
- Documentation
- Reports



Download Center

- Download PDF
- Download HTML
- Download JSON
- Download Markdown
- Download Executive Summary
- Download Refactor Plan
- Download Architecture Diagram



3. Scanner Features

File Traversal

- Walk full repository
- Skip .git
- Skip node_modules
- Skip venv
- Skip __pycache__
- Skip dist
- Skip build



Language Detection

Detect file language by extension and content.

Complexity Analyzer

Measures:

Cyclomatic Complexity
Cognitive Complexity
Function Length
Class Length
File Size



Threshold examples:

Complexity > 10
Function > 50 lines
Class/File > 250 lines



Code Smell Analyzer

Detects:

Long Function
Deep Nesting
Long Parameter List
God Class
God File
Dead Code
Unused Variables
Missing Error Handling
Duplicate Code



Security Analyzer

Detects:

Hardcoded Secrets
API Keys
Passwords
Tokens
eval()
pickle.loads()
shell=True
SQL Injection



Unsafe Subprocess
Dangerous Function Calls

Documentation Analyzer

Checks:

Missing Docstrings
Missing Comments
Undocumented Public APIs
Missing README Sections



Dependency Analyzer

Finds:

Circular Dependencies
Duplicate Libraries
Outdated Packages
Unused Packages
Dependency Version Conflicts



4. Architecture + Explorer Features

Architecture Explorer

Generates a visual graph:

Frontend
↓
API Layer
↓
Services
↓
Repositories
↓
Database



Interactive Code Explorer

User can click:

File
Class
Method
Function
Module



Then view:

Imports
Dependencies
Call Graph
Related Files
Function Details
Risk Score



5. Git + Risk Features

Git History Intelligence

Uses:

Commit Frequency
Bug Fix Count
Code Churn
Recent Changes
Bus Factor
Contributor Count



Risk Score Formula

Risk Score =
Static Quality
+ Churn
+ Bug Fixes
+ Recency
+ Bus Factor



Technical Debt Predictor

Predicts:

Future Problem Files
High-Risk Modules
Files Likely to Need Refactoring
Files with Growing Complexity



Risk Heatmap

Shows files as:

Low Risk
Medium Risk
High Risk
Critical Risk



6. Scoring Features

Quality Scoring Engine

Outputs:

Score 0-100
Letter Grade: A, B, C, D, F
Issue Count
Severity Count
Improvement Score



Example:

Grade: B+
Risk Score: 83
Security: 2 High
Technical Debt: 14
Coverage: 89%



7. AI Refactor Features

AI Refactor Skill

Detects:

Long Method → Extract Method
Nested If → Guard Clauses
Hardcoded Secret → Environment Variable
God Class → Split Class
Duplicate Code → Shared Utility
Unsafe Call → Safer Alternative



Refactor Lounge

Each finding becomes a card:

Issue
File
Severity
Risk
Explanation
[Refactor]
[Explain]
[Ignore]



One-Click Refactor

Refactor Function
Refactor File
Refactor All



8. Iteration Until Clean Skill

Main standout feature:

Scan
↓
Find Issues
↓
Generate Fix
↓
Apply Fix
↓
Validate
↓
Rescan





Repeat Until Clean

MCP/tool names:

```
fix_until_clean()  
improve_iteratively()
```



Goal:

Repeat until no critical errors or target grade is reached.



Example:

```
Before: Grade C  
After: Grade A-  
Improvement: +28%
```



9. Validation Features

After refactoring, run:

```
Lint  
Unit Tests  
Static Analysis  
Security Analysis  
Syntax Check  
Type Check  
Regression Check
```



Verify:

```
No Regression  
No New Critical Issues  
Build Still Passes  
Tests Still Pass
```



10. AI Agent / MCP Features

MCP Server Tools

Expose:

scan_project
scan_file
get_health_score
explain_finding
generate_refactor
fix_until_clean
download_report
list_analyzers
run_precommit_scan

Multi-Agent System

Agents:

Planner Agent
Scanner Agent
Risk Agent
Refactor Agent
Validation Agent
Documentation Agent
Report Agent

Flow:

Planner decides scan plan
Scanner runs analyzers
Risk Agent ranks issues
Refactor Agent suggests fixes
Validation Agent tests fixes
Documentation Agent writes docs
Report Agent generates output

11. Report Features

Developer Report

Includes:

Files Scanned
Issues Found
Severity Breakdown
Code Smells
Security Findings
Suggested Fixes



Manager Executive Summary

Includes:

Repository Grade
Top Risks
Business Impact
Recommendations
Estimated Improvement



CI/CD Report

Machine-readable:

JSON Output
Quality Gate Result
Fail/Pass Status



12. Pre-Commit Workflow

Purpose

Stop risky code before it enters Git history.

Flow

```
Developer edits code
↓
git commit
↓
pre-commit hook runs scanner
↓
Quick checks run
↓
If pass → commit allowed
```





If fail → commit blocked with report

Pre-Commit Checks

Secrets
Syntax
Complexity Threshold
Long Functions
Dangerous Calls
High-Severity Security Issues



Command

```
codepulse scan --staged --mode quick --fail-on high
```

Config

```
repos:  
  - repo: local  
    hooks:  
      - id: codepulse-scan  
        name: CodePulse AI Pre-Commit Scan  
        entry: codepulse scan --staged --mode quick --fail-on high  
        language: system  
        pass_filenames: false
```

13. Architecture Decisions

1. Client-Server Architecture

React Client + FastAPI/Flask Backend



Keeps UI separate from scanner logic.

2. Pure Scanner Core

Scanner core does not depend on Flask or FastAPI.

Reusable by:

Web App
CLI
MCP Server
Pre-Commit Hook
CI/CD



3. Plugin Analyzer Architecture

Each analyzer is separate:

ComplexityAnalyzer
SecurityAnalyzer
DependencyAnalyzer
GitRiskAnalyzer
DocumentationAnalyzer
RefactorAnalyzer



4. JSON-First Reporting

Generate JSON first, then render:

PDF
HTML
Markdown
Dashboard



5. Refactor Skill Separate from Scanner

Scanner finds problems.

Refactor skill fixes problems.

6. Validate Before Accepting Fixes

AI fixes must pass validation before being accepted.

14. Main Use Cases

1. Developer uploads repo/file and runs selected scans.
2. Developer drags and drops ZIP/file for quick analysis.
3. Developer selects only needed features.
4. Developer views code health dashboard.
5. Developer fixes high-risk findings with AI Refactor Skill.
6. Developer runs fix_until_clean loop.
7. Team blocks bad commits with pre-commit workflow.
8. Manager downloads executive summary.
9. CI/CD uses JSON report as quality gate.
10. Claude/Cursor uses MCP tools to scan and refactor code.



Final One-Line Story

CodePulse AI lets developers upload code, choose analysis features, detect quality/security/architecture risks, generate AI refactors, validate fixes, repeat until clean, enforce pre-commit quality gates, and download reports for developers, managers, and CI/CD.

